ReleaseProof

The firmware boots.
The fleet can't upgrade.

IN THE LAB · THE NEW FIRMWARE

✓ New firmware boots

ML-DSA bootloader + ML-DSA image
→ application running

Compatible

IN THE FIELD · THE UPGRADE

✕ Upgrade rejected

ECDSA bootloader: "Update type invalid"
→ version 1 keeps running

Incompatible

Same device. Same migration. Two different questions.

THE MISSING BRIDGE

Every update must get past the bootloader already in the field

DEPLOYED DEVICE

ECDSA bootloader

The gatekeeper. It checks the signature of every update, and it only knows today's algorithm.

THE TRANSITION

?

RARELY TESTED

NEW FIRMWARE

ML-DSA signed

Post-quantum signature. Built, signed and booted fine in the lab.

WHY NOW

Signatures like ECDSA are expected to fall to future quantum computers. NIST standardised ML-DSA in 2024, and firmware signing is starting to move.[1]

WHO FEELS IT

Industrial IoT makers, e.g. wireless condition-monitoring sensors, often installed where they are hard to reach. Our demo chip is used in such sensors.[2]

WHAT A MISSING BRIDGE COSTS

If the gatekeeper rejects the update and can't replace itself, the fix is manual: a technician, a cable, a visit, device by device.

Most tests check the destination. ReleaseProof tests the bridge.

[1] NIST FIPS 204 (ML-DSA), Aug 2024. [2] Nordic Semiconductor documents nRF52840 in NI MON-10411, Lynxemi and Atomation sensors: hardware relevance only, not their firmware and not customers.

LIVE DEMO

Let's test the bridge

1 Pick a configuration

Nordic nRF52840 + wolfBoot. Every commit and toolchain pinned.

2 Pick the change

ECDSA P-256 → ML-DSA-87 post-quantum signatures.

3 Run: it really executes

Real bootloader builds, fresh keys, boot and update in the Renode emulator, positive and negative tests.

42 s	8/8	61
BUILD + EMULATION	TESTS EXECUTED	HASHED ARTIFACTS

CAN THIS MIGRATION PROCEED AS PLANNED?

⊗ INCOMPATIBLE

Candidate	Compatible
Transition	Incompatible

X01 — **Deployed ECDSA bootloader receives the ML-DSA update** — Rejected

```
UART: "Update type invalid 0xB01≠0x200"
version 1 before and after reset
```

E11 — **Can the bootloader replace itself in the field?** — No

```
no self-update path on this port (RAM_CODE=0)
```

Decided by a deterministic engine. Every line links to a hashed artifact.

Historical LIVE run RP-20261017T124806Z-a39610, 17 Oct 2026. 42 s = build + emulation only, not end-to-end time with AI. Emulation, not silicon.

WHERE THE AI FITS

AI investigates. Evidence decides.

AMD-HOSTED AI

Plan

Adds checks from a registered catalog. Can't remove required ones.

→

RUNNER

Execute

Real builds and emulation. No network, every artifact hashed.

→

ENGINE

Decide

Same evidence, same verdict, same hash. No LLM involved.

→

AMD-HOSTED AI

Explain

Cites real evidence IDs. Proposes next steps from an allowed set.

→

ENGINEER

Choose

Picks a proposal. It becomes a new run; history never changes.

ORIGINAL RUN

INCOMPATIBLE

The deployed bootloader can't replace itself over the air.

→

engineer picks: "re-flash the bootloader in a service window"

NEW RUN · RE-EVALUATION

INDETERMINATE

Blocker removed, but recovery from an interrupted update was never observed. Not a pass.

An assistant built to please would say yes. ReleaseProof says: not proven yet.

- **Can't** change a verdict
- **Can't** remove required tests
- **Must** cite evidence that exists
- **AMD only**, no fallback provider

Inference: AMD Token Factory · Google Gemma-4-31B-IT (managed GPU undisclosed). Per-run traces in the app's AMD AI tab; if the endpoint is down the UI shows AI_UNAVAILABLE. Guardrails check citation IDs and verdict labels, not every sentence. Gemma acceptance runs 87d365 (wolfBoot) and 092e99 (MCUboot).

NOT A ONE-OFF

Different bootloader. Different blocker. Same engine.

NRF52840 + WOLFBOOT

The target works

...but the path is blocked

- ✓ ML-DSA-87 bootloader and image build and boot
- ✗ Deployed ECDSA bootloader rejects the ML-DSA update
- ✗ No bootloader self-update on this port

Candidate compatible **Transition incompatible**

`run a39610 · 8/8 tests · 61 artifacts`

NRF52840 + MCUBOOT (ZEPHYR 4.2)

The target can't be built

...and the path is blocked too

- ✗ Pinned signing tool refuses ML-DSA keys (measured by running it)
- ✗ Single-stage MCUboot can't replace itself in the field
- ✓ Tampered and wrong-key updates are rejected

Candidate incompatible **Transition incompatible**

`run 7682fc · 7/7 tests · 50 artifacts`

Shared test semantics **Runners measure, only the engine decides** **Not observed = UNKNOWN, never a pass**

WHERE THIS GOES

From one check to every release, for every device already shipped

TODAY · BUILT

Migration preflight

- 2 supported configurations, real builds + emulation
- Separate candidate and transition verdicts
- Portable evidence pack: manifest, hashes, UART
- Strict release-gate CLI: only a fully compatible migration passes

NEXT · VALIDATION

Closer to the field

- Hardware-in-the-loop on real dev kits
- Interrupted-update and power-loss tests
- More chips and bootloaders: STM32, NXP, commercial forks

VISION · THE FLEET MATRIX

Every generation, one answer each

DEPLOYED GENERATION	ROUTE TO NEXT RELEASE
HW rev A · bootloader 1	✕ **none: stranded**
HW rev A · bootloader 2	**via bridge release**
HW rev B · bootloader 2	✓ **direct**
HW rev B · bootloader 3	**? unknown**

Illustrative concept, not a current feature

For: firmware leads and product-security teams at industrial IoT manufacturers, who must keep shipped devices updatable for years.

Model: paid onboarding per product profile, then a subscription per profile. Not per device. HYPOTHESIS

Don't just test where you're going. Test whether you can get there.

ReleaseProof checks that the devices you already shipped can actually reach your next firmware, with executed evidence, before rollout.

Looking for design partners

Industrial IoT teams with a signing-key, bootloader or post-quantum migration on their roadmap. Bring one board and one change; we'll test the bridge.

Try it

Demo	`d1yxcnwgdgjzt6.cloudfront.net`
Code	`github.com/francescocarpineti091299/release-proof`

BACKUP · ARCHITECTURE AND AMD

AMD powers planning and explanations, not pass/fail decisions

Next.js UI
wizard · live run (SSE) · evidence · AMD panel

Caddy
TLS edge · /api → api · / → web

FastAPI
orchestrator · queue · SQLite · manifests

↓ internal network, no internet egress

runner-wolfboot
wolfBoot 4c79eee · wolfSSL 4aa1ad7 · Arm GCC 14.2.rel1 · Renode 1.16.1

runner-mcuboot
MCUboot 4eba8087 · Zephyr v4.2.0 · SDK 0.17.4 · imgtool 2.2.0 · Renode + own NVMC model

↓ typed observations + sha256 of every artifact

Adapters
missing observation ⇒ UNKNOWN + gap

V1.2 engine
validate · evaluate · explanation graph · canonical hash

AI gateway
planner · explainer · alternatives · assistant; schema, citation and verdict-phase checks

↓ persisted per run

Immutable run record
verdicts · dossier · manifest.json (inputs, pins, artifact hashes, result hash) · AI traces. Runner crash ⇒ FAILED, no verdict.

INFERENCE	AMD Token Factory, OpenAI-compatible, allow-listed host
MODEL	Google Gemma-4-31B-IT
GPU	Managed, undisclosed. Labels are not attestation
FALLBACK	None. Endpoint down ⇒ AI_UNAVAILABLE, deterministic plan runs
DEMO HOST	AWS EC2 c7a.2xlarge (AMD EPYC 4th gen) behind CloudFront, per infra audit of 17 Oct 2026
SMOKE	10/10 logical calls OK with Gemma. Not a benchmark

Scoped engineering evidence, stated honestly

Real today

- Upstream bootloader code at pinned commits, compiled with pinned toolchains on every run
- Ephemeral signing keys; real binaries booted and updated in Renode
- Negative tests: tampered image, wrong key, old bootloader receiving the new image
- Deterministic V1.2 engine with golden-hash regression tests
- Re-evaluations as new runs with a parent reference; history immutable
- Failure handling: runner killed ⇒ FAILED, no verdict, nothing replayed
- Evidence pack download and strict preflight CLI with engine replay

Not claimed

- Results on physical silicon: this is emulation
- Power-loss / interrupted-update safety: stays UNKNOWN (INV-I4)
- Support for arbitrary or proprietary firmware: 2 configurations only
- Certification or regulatory compliance: non-official assessment
- Semantic truth of every AI sentence: checks cover IDs and verdict labels
- Managed GPU identity, benchmark numbers, customers or ROI
- Hosted CI execution or OTA integration

Complementary to scanners and OTA, focused on the transition

Category · examples	Main documented focus	Relationship to ReleaseProof
Binarly · ONEKEY crypto inventory / PQC readiness	Binary analysis, crypto exposure; Binarly also documents reachability and CI gates	Complementary: a finding there is a natural trigger for a run here
Finite State adjacent release assurance	Evidence-linked AI, release policy gates	Closest adjacent; our focus is the deployed → candidate bootloader transition
Memfault · Mender · Foundries.io OTA / fleet lifecycle	Delivery, staged rollout, telemetry	Partners: they deliver, we pre-check the path
Renode · upstream bootloader CI building blocks	Repeatable emulation, same-scheme update tests	We add cross-generation tests, transition rules and evidence

Illustrative exposure, not ROI

5,000 sensors × 5% needing a visit × €250 = **€62,500**. Every input is hypothetical; a pilot should measure blockers found, hours saved and visits avoided.

Regulatory context, not a mandate

EU CRA: reporting from 11 Sep 2026, main obligations 11 Dec 2027. CNSA 2.0 (US national security systems only): firmware signing exclusive by 2030. NIST IR 8547 is still a draft.